

Project 1 - Escape Room Robot

Exploration, Mapping and Key Discovery

Platform: Elephant Robotics myAGV 2023 Pi (omnidirectional) - real robot, ROS 2

Team 02

Chatziparaskeva Anastasia 1104665

Pnevmatikaki Myrsini 1101023

Github repository: <https://github.com/myrsinipn/-Escape-Room-Robot-Exploration-Mapping-and-Key-Discovery.git>

Section 1. Introduction and Problem Statement

1.1 Project Overview

The objective of this project is to develop and deploy a fully autonomous robotic pipeline capable of navigating, mapping, and solving an unknown "escape room" style maze environment. Utilizing the **Elephant Robotics myAGV 2023 Pi**—an omnidirectional mobile robot platform equipped with a 2D LiDAR and an RGB camera—the system must explore a completely unmapped space without human intervention.

The task introduces a dynamic spatial puzzle: the environment contains restricted regions bounded by "doors," which are physically designated by 2 ArUco markers. A door is treated as a solid obstacle until the perception system discovers and localizes its corresponding "key" (represented by a separate, unique ArUco marker ID). While the logical mapping between key-door pairs is known a priori, the physical coordinates of these landmarks, doors, and maze walls must be discovered dynamically.

1.2 Scope of Challenges

While the high-level logic is straightforward, executing this behavior on physical hardware in real time introduces several complex sub-problems:

- **Perception:** Robustly detecting and computing the spatial coordinates of structural boundaries, ArUco markers and corner landmarks.
- **State Estimation:** Minimizing odometric drift and localizing the robot within a growing map using sensor fusion.

1.3 System Structure

To manage these challenges, the problem is decomposed into the four core pillars of a autonomy stack: **Sensing, Perception, State Estimation (SLAM), Exploring** and **Decision Making**.

To maximize computational throughput on the embedded processor, the entire pipeline is implemented as a collection of modular **ROS 2 nodes** executing concurrently within a singular multi-threaded executor. This architectural choice prevents critical operations—such as the Extended Kalman Filter (EKF) update loop, sensor callbacks, and path planners—from blocking one another.

Section 2. System Architecture

2.1 Execution Model and Concurrency

The software architecture is designed as a modular, decentralized pipeline. The entire ROS 2 autonomy stack is initialized via a central entry point (`main.py`), which registers all sensor interfaces and core computational nodes to a single `MultiThreadedExecutor`.

Because the robot must react to real-time physics, managing concurrency is critical. We utilized a `ReentrantCallbackGroup` to allow multiple callbacks to execute in parallel. This highly concurrent design necessitated strict thread-safety measures. Without mutual exclusion (`threading.Lock`), severe race conditions would occur between the mapping and exploration threads. To prevent memory corruption, all shared spatial structures are guarded by dedicated locks and use copy-on-write mechanisms.

2.2 Hybrid ROS Interface and System Launch

While our custom autonomy stack is built on **ROS 2 Galactic**, the native hardware drivers provided for the myAGV 2023 Pi—including the chassis odometry (`myagv_active`) and the YDLidar (`ydlidar_ros_driver`)—operate on **ROS 1 Noetic**.

To facilitate seamless communication across this middleware boundary, we implemented a hybrid architecture using the `ros1_bridge`.

- **The Parameter Bridge:** We utilize a `parameter_bridge` configured via a custom `bridge.yaml` to explicitly map critical topics between the two environments. The bridge successfully translates hardware states to the ROS 2 layer (`/scan`, `/odom`, `/imu`, `/tf`, `/tf_static`) and passes motion commands back down to the ROS 1 base (`/cmd_vel`).
- **Sequential Bring-Up:** Because the bridge requires the ROS 1 master and topics to be fully initialized before mapping, starting the system concurrently would lead to orphaned topics or crashes. To solve this, the deployment relies on a custom bash script utilizing `tmux` to enforce a strict, sequential launch procedure. The script pauses execution at each step—verifying `roscore`, base odometry, and LiDAR are actively publishing—before deploying the parameter bridge and finally launching the native ROS 2 camera node (`v4l2_camera`).

2.3 Node Topology and Data Flow

The active components of the ROS 2 system are divided into specialized nodes communicating asynchronously:

- **Sensor Abstraction Nodes:** To decouple the core logic from raw hardware drivers, lightweight nodes (`LidarSensor`, `CameraSensor`, `OdometrySensor`) subscribe to the bridged topics and cache the latest messages.

- **EKFLidarSLAM:** The central state estimation authority. It consumes odometry and processed LiDAR data to maintain the robot's global pose and construct the log-odds occupancy grid.
- **ArucoMonitor:** The visual perception node evaluating the camera feed to detect, classify, and compute the relative poses of ArUco markers.
- **RRTEExplorer:** The integrated mapping and control node. It actively reads the SLAM-generated occupancy grid to identify unmapped frontiers, plans paths, and generates velocity commands to drive the robot.

2.4 Distributed Visualization and Docker Integration

To preserve the Raspberry Pi's computational resources strictly for real-time autonomy tasks (SLAM, perception, and control), all graphical rendering and user monitoring are offloaded to a remote operator laptop. The robot and laptop communicate over a local Wi-Fi network using matching ROS_DOMAIN_ID configurations to isolate the ROS 2 DDS traffic.

The visualization pipeline relies on several dedicated helper scripts to bridge the raw data into standard ROS 2 monitoring tools:

- **Coordinate Frame Broadcasting (TF2):** The custom SLAM output is not natively tracked by the standard ROS TF tree. To resolve this, a dedicated script (`slam_pose_to_tf.py`) executes on the robot to continuously translate the EKF-SLAM pose output into standard tf2 transforms (bridging the map frame to `base_link`).
- **URDF and State Publishing:** On the remote laptop, the physical dimensions and visual meshes of the robot are loaded by parsing the `myagv_plus.urdf.xacro` file using the `xacro` utility. This URDF ([urdf](#)) is fed into the `robot_state_publisher`, which broadcasts the static transforms of the chassis.
- **Dynamic Wheel Animation:** To provide accurate visual feedback of the omnidirectional kinematics within RViz, a custom script (`slam_wheel_animator.py`) is run alongside the `joint_state_publisher`. This dynamically animates the Mecanum wheels based on the robot's physical movement.

Cross-Distribution Docker Environment A significant system integration challenge arose due to operating system constraints: the operator laptop runs natively on **ROS 2 Humble**, while the myAGV base is strictly bound to **ROS 2 Galactic**. To prevent DDS middleware incompatibilities between the two distributions, RViz2 is executed via containerization.

The operator laptop spins up an `osrf/ros:galactic-desktop` Docker container ([Docker](#)) utilizing the `--network=host` flag to capture the robot's domain traffic. To render the graphical user interface out of the container and onto the host machine, X11 server forwarding is established by mapping the `.X11-unix` socket (`--volume /tmp/.X11-unix:/tmp/.X11-unix`) and passing the host's `DISPLAY` environment variables into the Docker instance. Once launched, RViz2 renders the live map, ArUco spheres, and the loaded URDF model seamlessly.

Section 3. Sensors and Preprocessing

To maintain a strict boundary between the ROS 2 middleware and our core algorithmic logic, each physical sensor is encapsulated within its own dedicated, lightweight ROS 2 node. These sensor nodes (`LidarSensor`, `OdometrySensor`, `CameraSensor`) subscribe to their respective topics and stash the latest messages behind thread-safe getters. This design ensures that the rest of the autonomy stack never interacts with raw ROS message types directly, making the system decoupled and allowing for isolated unit testing of the algorithmic components via sensors.

3.1 LiDAR Sensing and Scan Preprocessing

The `LidarSensor` reads polar coordinate arrays from the `/scan` topic. As implemented, this node is purposefully maintained as a thin data-ingestion layer. During the ROS callback, it dynamically constructs a synchronized angles array utilizing the message's minimum, maximum, and increment bounds. It manages initial data integrity by truncating any length mismatches between the range and angle arrays, and it rapidly masks any raw NaN or inf values, replacing them with a predefined maximum safe range (10.0 m). To ensure thread safety when concurrent nodes request the sensor data, the getter method strictly yields decoupled copies of the NumPy arrays.

Once safely ingested, the arrays are passed to the stateless `ScanPreprocessor`, which takes ownership of the spatial filtering. The preprocessing pipeline executes three sequential operations:

1. **Range Clipping:** Measurements are tightly constrained to a reliable working window to filter out self-collisions with the robot chassis and exclude the 10.0 m replacement values assigned to invalid rays.
2. **Signal Smoothing:** To prevent the corner-extraction algorithms from falsely triggering on single noisy beams, the ranges are smoothed using a small moving average kernel of size 5.
3. **Coordinate Transformation:** The preprocessor transforms the cleaned polar scan r_i, θ_i into Cartesian coordinates x_i, y_i . This Cartesian matrix is the required

format for both the EKF landmark extractor and the log-odds occupancy grid update.

3.2 Odometry Sensing and the Motion Model

The OdometrySensor node isolates the core autonomy stack from the complex nav_msgs/Odometry message structure. Subscribing to the /odom topic, the callback extracts the positional coordinates x,y and utilizes tf_transformations.euler_from_quaternion to collapse the 3D orientation quaternion into a single planar yaw angle (). Simultaneously, it extracts the body-frame twist velocities (vx,vy,ω). Consistent with the system's thread-safety protocols, both the pose and velocity states are stored as NumPy arrays and yielded to the requesting nodes via strict .copy() operations.

Because the myAGV utilizes a four-wheel Mecanum chassis, standard differential-drive kinematics are insufficient. The raw body velocities provided by the sensor node are passed to a custom, stateless OmniMotionModel that accurately integrates them into the world frame:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

Crucially, this model also computes and returns the state transition Jacobian F with respect to the robot's pose, which is mathematically required by the EKF for the covariance prediction step.

During physical deployment, we identified a critical hardware calibration anomaly: a sign inversion between the odometry's reported yaw rate and the expected cmd_vel rotational axis. If left uncorrected, the opposing data causes the robot to spin aggressively in place as the path follower fights the odometry estimation. To resolve this handedness mismatch, a configurable yaw_sign parameter was integrated into the motion model to dynamically invert the angular velocity coordinate frame before integration.

3.3 Camera Sensing and Visual Interface

The CameraSensor node serves as the dedicated visual ingestion layer, subscribing to the uncompressed /camera/image_raw topic. Upon receiving a ROS sensor_msgs/Image payload, the node utilizes the cv_bridge library to seamlessly decode the message into a standard OpenCV-compatible bgr8 NumPy matrix.

Consistent with the architecture's concurrency protocols, the getter method strictly yields decoupled .copy() instances of the most recent frame. This guarantees that downstream computer vision algorithms can aggressively process the image array

without risking memory corruption from the asynchronous ROS subscriber thread continuously updating the buffer.

To ensure spatial accuracy during subsequent ArUco 3D pose estimation, the camera's intrinsic matrix (K) and distortion coefficients (D) are loaded at runtime from a localized config/camera_calibration.json file. By design, this high-bandwidth camera stream is explicitly routed only to the visual perception node (responsible for classifying semantic "doors" and "keys") and is strictly excluded from the geometric EKF-SLAM pipeline, thereby preserving critical computational bandwidth on the Raspberry Pi.

Section 4. EKF-LiDAR SLAM

Extended Kalman Filter - Algorithm

- 1 Input: $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$
- 2 $\bar{\mu}_t = f(\mu_{t-1}, u_t)$
- 3 $\bar{\Sigma}_t = F_t \Sigma_{t-1} F_t^T + R_t$
- 4 $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$
- 5 $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$
- 6 $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$
- 7 Output: μ_t, Σ_t

- We use the non-linear functions
- We use F_t instead of A_t
- We use H_t instead of C_t

The mathematical core of the robot's spatial awareness is the EKFLidarSLAM node. It executes a landmark-based Extended Kalman Filter (EKF) utilizing geometric corners extracted from the LiDAR scan as static map features.

The system state vector at time-step k continuously expands as new landmarks are confirmed. It is defined as:

$$\mathbf{X}_k = [x_k, y_k, \theta_k, m_{1,x}, m_{1,y}, \dots, m_{n,x}, m_{n,y}]^T$$

where x_k, y_k, θ_k represents the robot's pose in the global frame, and $m_{i,x}, m_{i,y}$ represents the stationary 2D Cartesian coordinates of the i-th confirmed map corner.

4.1 Kinematic Prediction

On every odometry callback (running at 20 Hz), the prediction step integrates the omnidirectional body-frame velocities (v_x, v_y, ω). To preserve linearization accuracy over larger time intervals, the integration is split into sub-steps of at most 0.05 s. The covariance is propagated using the linearized error dynamics:

$$\Sigma_k|_{k-1} = F_k \Sigma_{k-1}|_{k-1} F_k^T + R_k$$

where R_k is the motion noise matrix (diagonal, with $\sigma = 0.02$ m in x/y and $\sigma = 1.5^\circ$ in heading), and the state transition Jacobian F_k is structured so that only the top-left 3×3 robot block is updated, as stationary landmarks do not move during prediction:

$$F_k = \begin{bmatrix} F_{robot} & \mathbf{0}_{3 \times 2n} \\ \mathbf{0}_{2n \times 3} & I_{2n \times 2n} \end{bmatrix}$$

4.2 Feature Extraction via RDP and Arm Verification

Raw LiDAR scans are highly susceptible to noisy spikes that can be misclassified as corners. The feature extraction pipeline uses a robust, distance-invariant polyline method:

RDP Segmentation: The ordered LiDAR points are first clustered into continuous segments based on range gaps (gap threshold: 0.20 m). Each segment is then simplified using the iterative Ramer-Douglas-Peucker (RDP) algorithm ($\epsilon = 0.04$ m). Segments with fewer than 6 points are discarded before RDP is applied. The vertices of the simplified polylines form the initial corner candidates.

Arm Length Verification: To reject noise spikes (e.g., a single errant laser return on a flat wall), the physical length of both arms extending from each candidate vertex is computed. Both arms must exceed a minimum length of 0.15 m, and the interior angle must lie within the range $[85^\circ, 95^\circ]$ to be classified as a valid physical corner.

Deduplication: Observations whose projected world positions lie within 0.70 m of an already-accepted observation in the same scan are discarded.

4.3 Data Association and the Candidate Buffer

To prevent a single spurious detection from permanently polluting the state vector, unassociated observations are routed to a temporary candidate buffer.

Data Association: For each extracted corner, the predicted measurement $\hat{\mathbf{z}}$ (range and bearing) to every known landmark is computed via the observation model. The innovation $\mathbf{v} = \mathbf{z} - \hat{\mathbf{z}}$ is evaluated using the Mahalanobis distance:

$$D_m^2 = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}$$

where

$$S = H \Sigma H^T + Q$$

A match is accepted if $D_m^2 < 5.99$ (95% confidence interval for 2 DOF), the raw range error is < 0.5 m, and the bearing error is $< 20^\circ$.

Promotion Criteria: If an observation fails the gate, it is tracked as a candidate, with its position updated as a running weighted average. A candidate is promoted to the permanent EKF state vector if it is observed ≥ 4 times, or ≥ 3 times with a spatial variance below 0.08 m. New landmarks are initialised with a position uncertainty of $\sigma = 0.28$ m per axis.

4.4 EKF Correction

For successfully associated landmarks, the state is updated using the standard EKF equations. The Kalman gain is:

$$K = \Sigma H^T S^{-1}$$

The state and covariance are then updated as:

$$\begin{aligned}\mu_k &= \mu_{k|k-1} + K v \\ \Sigma_k &= (I - K H) \Sigma_{k|k-1}\end{aligned}$$

As a safety net, the proposed correction is evaluated before being applied. If a single update would shift the robot's XY position by more than 5 cm or its heading by more than 3° , the correction is discarded entirely, as such large jumps indicate a data association failure rather than a valid state update.

4.5 Occupancy Grid Mapping and Concurrency

In parallel with the landmark filter, the node maintains a dense log-odds occupancy grid at 5 cm/cell resolution over a 30×30 m area. For every scan, rays are traced from the robot's cell to each LiDAR endpoint, decrementing the log-odds of intermediate cells (free space, $\Delta = -0.9$) and incrementing the endpoint cell (occupied, $\Delta = +1.4$), clamped to $[-5.0, +10.0]$. Every second ray is used to reduce computational load on the hardware.

Deploying this on physical hardware required three engineering solutions:

Thread Synchronization: The grid arrays are written by the map-update callback and read concurrently by the 2 Hz map publisher. Access is guarded by a `threading.Lock()` (`map_lock`). A dedicated `_scan_lock` decouples the scan stash written by the 5 Hz correction step from the map publisher, preventing the publisher from reading a partially updated scan array.

Decoupled Raycasting: Raycasting is performed inside `publish_map_step()` (running at 2 Hz) rather than inside `correction_step()`. This prevents the computationally expensive

grid update from blocking the 20 Hz EKF prediction timer and starving the localization pipeline.

Rotation Freezing: Omnidirectional robots experience map smearing when pivoting in place due to timing delays between odometry and LiDAR. If the angular velocity from odometry exceeds 0.18 rad/s while linear speed remains below 0.01 m/s, all raycasting updates are suspended until the rotation completes, preserving map fidelity during turning manoeuvres.

4.6 Debugging Methodology: EKF-SLAM Validation Scripts

Three scripts were used to validate EKF-SLAM independently of the full stack (RRT explorer + ArUco monitor), alongside earlier manual teleoperation testing with a PS2 gamepad. Removing the planner and marker monitor eliminates every other writer to `/cmd_vel`, so any pose or trajectory anomaly observed can be attributed to SLAM itself rather than to interference from other nodes.

1. debugslam.py - Straight-Line SLAM Test

debugslam.py runs EKF-SLAM live against a known, simple trajectory, so that any curvature, drift, or jump in the SLAM output is immediately diagnostic rather than something that has to be inferred from a more complex path. Its StraightCmdTest node publishes a Twist message with `linear.x=0.08` on `/cmd_vel` continuously, driven by a 20 Hz ROS timer, producing pure open-loop straight-line motion with no turning and no operator input. `main()` wires LidarSensor, OdometrySensor, ScanPreprocessor, and OmniMotionModel into EKFLidarSLAM using the constructor-injection pattern, then adds all four nodes-lidar, odom, straight_test, and slam-to a single MultiThreadedExecutor and spins them together. On shutdown, `stop_robot()` flushes ten zero-Twist messages to guard against publish loss, and `slam.print_debug_summary()` dumps a final diagnostic summary before the nodes are destroyed.

Because the commanded path is known to be perfectly straight, a healthy SLAM estimate should read as a straight line as well; any bend in the corrected pose points directly at a problem in the estimator rather than in the motion being tested.

2. debugplot.py - Live Trajectory Plot

debugplot.py is a read-only, non-actuating listener that visualizes `/odom` against `/slam_pose` in real time, and is intended to run alongside either test. Its DebugPlotNode subscribes to `nav_msgs/Odometry` on `/odom` and `geometry_msgs/PoseWithCovarianceStamped` on `/slam_pose`, appending each incoming (x, y) position to a pair of parallel lists under a `threading.Lock`, since the ROS callback thread and the plotting thread run concurrently; `get_data()` then returns a locked snapshot of both trajectories so the plotting side never reads a list mid-update. The LivePlot class draws raw odometry as a dashed blue line and the EKF-corrected

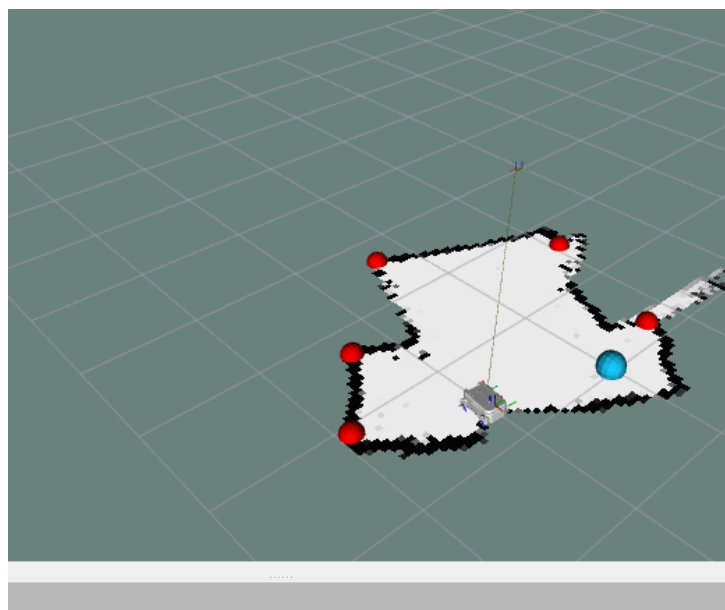
estimate as a solid red line, and sets the axes to an equal aspect ratio so that a metre in x is rendered the same length as a metre in y - without this, a turn's true angle would appear visually distorted on the plot.

Section 5. Exploration and Navigation

Exploration logic in this project is implemented through the RRTExplorer node, which runs two independent timers on a ReentrantCallbackGroup: a planner timer (every 3 s) that grows the RRT tree toward the nearest unmapped frontier and publishes the resulting path to RViz, and a control timer responsible for physical motion. Because reliably following RRT waypoints on the real robot proved difficult due to SLAM pose lag and odometry drift during in-place rotation, the system was split into two distinct operational configurations — two separate entry points, `main.py` and `main2.py` — each making a different trade-off between planning sophistication and motion reliability.

5.1 Map snapshot and the semantic costmap

Each planning cycle starts with `get_map_snapshot()`, which reads the EKF-SLAM log-odds grid and known-cell mask, thresholds them into occupied/free/unknown, and inflates obstacles by `robot_radius_cells` (5 cells = 0.25 m) using a circular structuring element via `scipy's binary_dilation`. Locked doors reported by the ArucoMonitor (`block_door_in_costmap`) are stored in `_locked_doors` as line segments and rasterized into this snapshot on every call via `_rasterize_segment()`, so a door blocks the planner the instant both of its markers are localized, and disappears the instant `unblock_door()` is called after its key is found. This costmap is rebuilt from scratch each cycle, so unlocking is immediate and leaves no stale occupied cells.



5.2 RRT Planning and Frontier Selection

The function `build_path()` grows a tree from the robot's current pose for up to 1000 iterations. With 30% probability a new sample is drawn near an already-found frontier; otherwise it is drawn uniformly in a disk of radius 1.35 m around the robot. Each candidate node is connected to its nearest existing node with a fixed step size of 0.50 m, and is kept only if both the node and the connecting edge are collision-free against the inflated costmap (`point_safe` / `line_safe`). A node is recorded as a frontier candidate if any cell in its 3×3 neighbourhood is unknown (-1) and its distance from the robot falls inside [0.75 m, 1.35 m]. Among the found frontiers, the planner does not simply pick the nearest one — it scores each by $|\text{distance} - 1.0 \text{ m}|$ plus a small heading-change penalty, preferring a short, forward-facing goal over a distant one, then re-validates the chosen frontier against a fresh map snapshot before returning the path. The resulting path is published on `/exploration_path` and the goal marker on `/rrt_goal`, making both visible in RViz as the robot moves through the room.

Rapidly exploring Random Trees (RRT)

RRT Algorithm:

```
1 : Initialize search tree with  $\mathbf{x}_{\text{start}}$ 
2 : while some stopping criteria is not met
3 :   sample  $\mathbf{x}_{\text{sample}} \sim \mathcal{X}$ 
4 :   find  $\mathbf{x}_{\text{nearest}}$  nearest node of  $\mathbf{x}_{\text{sample}}$  in tree
5 :   connect  $\mathbf{x}_{\text{nearest}}$  to  $\mathbf{x}_{\text{new}}$  in direction of  $\mathbf{x}_{\text{sample}}$ 
6 :   if success
7 :     add  $\mathbf{x}_{\text{new}}$  to the tree with an edge from  $\mathbf{x}_{\text{nearest}}$ 
8 :   if  $\mathbf{x}_{\text{new}} \in \mathcal{X}_{\text{goal}}$  return SUCCESS
9 : return FAILURE
```

Section 6. Perception: Keys and Doors,

The perception subsystem bridges the gap between semantic visual identification and spatial mapping. It is responsible for detecting ArUco markers, estimating their 3D poses, projecting them into the 2D global map, and managing the logical rules that dynamically update the navigation costmap.

6.1 Marker Detection and Pose Estimation

Visual processing is isolated within the `ArucoMonitor` node, which samples the camera frame at a throttled rate of 5 Hz (a 0.2-second timer) to conserve CPU cycles. To prevent

the same camera frame from being counted multiple times toward detection confirmation, the node compares the timestamp of each incoming frame against the last processed timestamp and skips duplicates.

For each new frame, it invokes the ArucoDetector, which first converts the image to grayscale and then runs `cv2.aruco.ArucoDetector` using the `DICT_4X4_50` dictionary and default detector parameters. Upon successful detection, the pipeline calls `cv2.aruco.estimatePoseSingleMarkers` to compute, for each detected marker, the rotation vector `rvec` and the translation vector `tvec = [x_cam, y_cam, z_cam]^T` relative to the camera's optical centre. A full homogeneous transformation matrix T_{cm} (marker frame $\rightarrow$ camera frame) is also constructed via the Rodrigues formula, though the primary localization path relies on `tvec` directly. In the standard OpenCV camera frame the x-axis points right, the y-axis points down, and the z-axis points forward.

Confirmation gating. To avoid acting on a single spurious detection, every marker must be detected in at least `confirmation_frames` consecutive unique frames (default: 3, loaded from [config/key_doors.json](#)) before it is treated as confirmed. Per-marker counts are tracked in `_detection_counts`; only confirmed markers proceed to localization and registry updates.

6.2 Spatial Localization (Camera to World Frame)

When a marker is confirmed, its coordinates exist strictly within the local 3D camera frame. The `DoorLocalizer` module is responsible for transforming this local coordinate into a stable global 2D world coordinate matching the SLAM map.

The camera is mounted with physical offsets $dx = 0.13$ m, $dy = 0.0$ m relative to the robot's base footprint. Because the OpenCV z-axis points forward and the x-axis points right, the projection onto the ground plane maps as follows:

$$x_{rob} = z_{cam} + d_x$$

$$y_{rob} = -x_{cam} + d_y$$

Once converted to the robot's local frame, the point is transformed into the global world frame using the current SLAM pose (x_r, y_r, θ_r) :

$$\begin{bmatrix} x_w \\ y_w \end{bmatrix} = \begin{bmatrix} x_r \\ y_r \end{bmatrix} + \begin{bmatrix} \cos \theta_r & -\sin \theta_r \\ \sin \theta_r & \cos \theta_r \end{bmatrix} \begin{bmatrix} x_{rob} \\ y_{rob} \end{bmatrix}$$

Position averaging. Camera pose estimates jitter frame-to-frame. To prevent a door boundary from jumping around the map (and leaving stale blocked-door overlays), the `DoorRegistry` maintains an incremental running mean over all observations of each marker. Every new observation is blended with the prior average weighted by

observation count, so the estimated world position converges smoothly without storing the full history.

These transformed global coordinates (x_w, y_w) are published to the `/aruco/markers_viz` topic as a `MarkerArray` — an orange sphere of diameter 0.15 m at ground level together with a white `TEXT_VIEW_FACING` label ("ID <n>") floating 0.3 m above it — providing live tracking in RViz.

6.3 Configuration (key_doors.json)

The key–door mapping is loaded entirely from [config/key_doors.json](#) at startup. The file specifies which pairs of ArUco IDs constitute each door, which key ID unlocks each door, and the required confirmation frame count:

```
{
  "door_marker_pairs": [{"door_id": 1, "markers": [5, 6]},
  "key_to_door": {"10": 1},
  "confirmation_frames": 3
}
```

This means marker IDs 5 and 6 form the boundary of Door 1, and marker ID 10 is its key. The loader validates that door IDs are unique, that no marker ID is reused across doors, that no key ID is also a door-marker ID, and that every key references a defined door.

6.4 Logical Registries and Costmap Management

Two registry classes separate the concerns of spatial bookkeeping and logical state:

- `DoorRegistry` (perception layer): stores the (markerA, markerB) → door_id mapping, maintains the running-mean world positions of all observed markers, computes each door's left, right, and center world coordinates once both of its markers are localized, and delegates key/door lock logic to `KeyDoorRegistry`.
- `KeyDoorRegistry` (state estimation layer): owns the key_to_door and door_to_key mappings, tracks the set of discovered keys, and exposes queries such as `is_door_unlocked(door_id)`, `get_locked_doors()`, and `get_unlocked_doors()`.

The `ArucoMonitor` drives four distinct actions based on what is detected each cycle:

1. Door boundary found – once both markers of a door are localized and the door is not yet unlocked, `explorer.block_door_in_costmap(left, right, door_id)` is called. This inserts the segment between the two markers as a solid obstacle into the

RRT costmap, preventing the path planner from routing through it.

A `_blocked` flag on the door entry prevents repeated calls.

2. Key found – when a confirmed marker ID matches a key in the `key_to_door` map, `DoorRegistry.register_key()` is called. On the first detection (`is_new_key = True`), the monitor triggers `explorer.unblock_door(left, right, door_id)`, reclassifying that region as passable free space. It then immediately calls `explorer.navigate_to_door(door_id, center)` to pre-empt the current exploration goal and drive the robot toward the newly opened passage.
3. Key seen before door is mapped – if a key is confirmed but neither marker of its corresponding door has been localized yet, the key is stored in `key_world_positions` without triggering an unblock. A second pass at the end of every detection cycle checks all known door positions and handles this deferred case: if a door is now unlocked and has not yet been unblocked, the unblock and navigation goal are issued then.

Door seen before its key – the door is blocked in the costmap as described in (1).

Once the key is subsequently confirmed, (2) or (3) above fires and opens it.

Section 7. Motion Control: Two Operational Configurations

During development it became clear that connecting the RRT planner's waypoint output directly to the robot's motion controller was not yet reliable enough for a live demonstration. The two-phase follower implemented in an earlier version of the explorer — first rotating in place until aligned with the next waypoint (TURN phase), then driving straight toward it (DRIVE phase) — works correctly in isolation, but on the physical robot the combination of SLAM pose lag, wheel-slip during rotation, and the 3-second replanning interval causes the robot to overshoot waypoints and occasionally drive into walls. To keep the demonstration safe while still showing the full planning pipeline, the system was split into two configurations.

Configuration 1 — `main.py` (presented first). The RRT planner runs normally: every 3 seconds it takes a map snapshot, grows a tree toward the nearest frontier, and publishes the path and goal marker to RViz. Physical motion, however, is handled entirely by the `SafeLidarMotion` node — a dedicated reactive controller that reads seven LiDAR sectors at 10 Hz and steers the robot away from obstacles with no dependence on the SLAM pose or planned path. The path-follower and control timer inside `RRTExplorer` are commented out so they cannot conflict with `SafeLidarMotion`. The result is a robot that roams the room safely while we can watch the RRT tree expand toward unexplored frontiers in RViz in real time. This corresponds to the standard global-planner / local-planner separation used in production mobile-robotics stacks: the global planner (RRT) computes where to go, and the local planner (`SafeLidarMotion`)

handles how to get there without hitting anything.

Configuration 2 — `main2.py` (shown as work-in-progress). This entry point uses `rrt_explorer2.py`, which includes the two-phase waypoint follower and drives the robot directly via `/cmd_vel`. No `SafeLidarMotion` is running; the RRT explorer is the sole source of motion commands. The semantic costmap updates from the ArUco pipeline — door blocking, key detection, unblocking, and priority navigation — are all active and affect both the planned path and the motion commands. This configuration demonstrates the intended end-to-end pipeline: plan a path to a frontier, follow it, detect a door, block it, find the key, unblock the door, and drive through. Path tracking on the physical robot remains sensitive to drift and is the primary area of ongoing improvement.